

Natural Language Processing Clinic Note Project

Josu Viteri
Gotzon Viteri
Iker Dominguez

2026/01

1 Overview

This project aims to act as a Natural Language Processing (NLP) lab where we will explore the different techniques of feature extraction, the usage of those features in tabular/shallow machine learning, custom deep learning and fine tuning of hugging face LLM and SLM, alternating between encoder only, encoder/decoder and decoder only models. The tasks we approached include automatic generation of clinical summaries, clinic dialog classification and clinic support generation. We used a dataset of approximately 1,700 conversations between doctors and their patients, along with their respective summaries and annotations. In respect to the dataset analysis and preprocessing, the goal is to verify the dataset's integrity, investigate its characteristics/features in a context and non contextual way, and ultimately train and validate models capable of the mentioned tasks.

Key Tasks:

First Delivery: Project proposal and approach overview.

Second Delivery: Dataset exploratory analysis, preprocessing and feature extraction

Third Delivery: Shallow machine learning and custom deep learning model training validation and testing.

Fourth Delivery: Hugging face model partial and complete fine tuning.

Each deliverable contains corresponding corrections to previous work.

1. Medical Dialogue Classification: Categorizing clinical conversations into predefined section headers. The dataset dialogs include labels for their categories, which allow supervised training approaches.

2. Clinical Conversation Summarization: Generating concise medical summaries from doctor-patient dialogues. Each dialog contains a short summary of the theme, which allows us to check the accuracy of our results.

Each task combines the application of different machine learning algorithms and techniques as mentioned, except for the “extra”, which only applies the fine tuning of a hugging face SLM.

3. Clinical Generative Support (Extra): Decoder Only SLM Fine-Tuning for generative clinical support, for personal pipeline deployment.

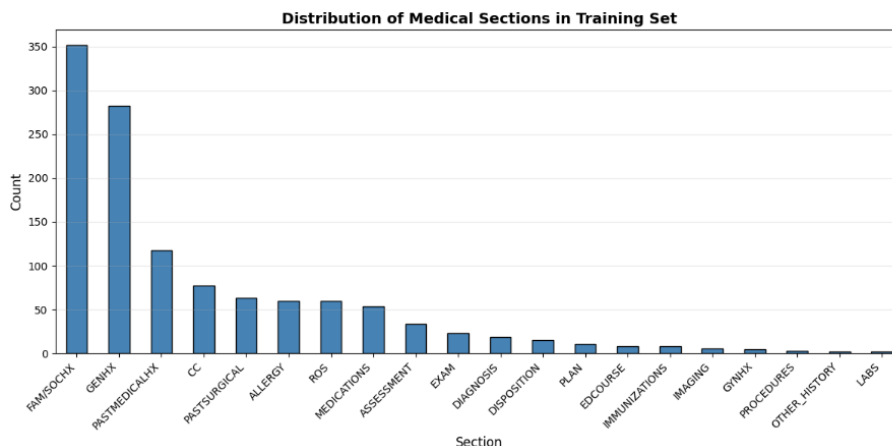
First Delivery: Project Proposal

Overall, the dataset was composed of 1700 conversations between doctors and patients, divided in training and validation and test sets.

Initially we aimed to develop a NLP model focussed on autonomous summarisation of clinical dialogs. Even though we had a clear idea of the objective of our project, the main challenge was to actually decide what to do with the dataset we had hosted at github, MTS Dialog dataset.

This dataset was published alongside the “An Empirical Study of Clinical Note Generation from Doctor-Patient Encounters “ paper. It was developed by Asma Ben Abacha, Wen-wai Yim, Yadan Fan and Thomas Lin during 2023, and acted as the groundwork for many projects that represent the state of the art. These initiatives mainly searched for efficient ways of approaching the task from machine learning, developing consistent usability of summaries generated by LLMs, in clinical contexts.

The inclusion of class headers facilitated supervised learning, therefore we established accurate clinical dialog classification as our second primary objective.



As we can appreciate in the plot, we have an easily noticeable class im-

balance, which ended up causing problems when the only training dataset was this one, as the models learn to classify the dialogs as the ones they have seen most, but was solved with transformers-attention based models, as we will see in next chapters.

2 Second Delivery: Dataset Analysis and Feature Extraction

The objective of the second deliverable was to run a proper analysis and preprocessing of the dataset. Once the dataset was well understood and the data was ready to be used, we extracted the features. These features allow the models to understand the text and be able to use the information available to learn and be ready for what they are going to be exposed to deal with. The features extracted include contextual and non contextual embeddings.

At the EDA, we learnt about the length of the dialogs, the classes available and the imbalance of their distribution.

During the dataset preprocessing we cleaned and normalized the text, generating non-textual encoding, tokenization, compared lemmatization and stemming (which are similar but differ in their context knowledge as lemmatization needs to understand it and generates different results). We continued with non contextual word embeddings and compared Word2Vec, Glove and FastText. The generated embeddings led us to understand the relevance of the words understood by each vectorizer model and the preprocessing previous to it.

The next step included the usage of TF-IDF, which works by detecting relevant words. TF-IDF is based on the principle that a word is statistically more significant if it appears frequently within a specific document but infrequently across the entire corpus.

Finally we loaded models in order to visualize the generated embeddings and run a principal component analysis (PCA).

3 Third Delivery: Shallow ML and Custom Deep Learning Architectures

The objective of the third deliverable was to correct the issues from the previous deliverable and to approach the problem using traditional shallow machine learning techniques and developing our custom deep learning based

neural networks for both summarisation and classification tasks. Then, evaluating the results with the respective metrics and testing the models with our test sets.

3.1 Corrections from second submission

First, we modified the previous submission to align with the suggestions received. We altered the method for creating both contextual and non-contextual embeddings. Overall, we realized that the high volume of unique and total tokens is not a problem as we had initially thought. Although we focused heavily on this aspect, it helped us understand that this is not a defect in our tokenisation, but rather a characteristic feature of our dataset.

3.2 Shallow ML classification

Regarding the classification task, we started using TF-IDF and Count Vectorizer with various classical machine learning classification techniques and compared them. The goal was to classify a conversation between a patient and their doctor (along with other potential participants) into a specific class.

Then we continued by fine tuning the classification head of Clinical-BERT to apply convolutional neural network techniques to our classification task. The model is not in the repository as it is too voluminous, but we have attached an example of the model operating in inference using the first conversation of the validation dataset.

This approach was not what we initially decided to do, a custom bidirectional LSTM based model, but we ended up completing it at the next submission.

3.3 Shallow ML summarisation

At the summarisation side, we applied classical machine learning techniques: TF-IDF, TextRank, and evaluation/comparison using ROUGE metrics. These techniques are based in tabular or shallow named machine learning.

The implementation of deep learning architectures was applied using Convolutional Neural Networks with an LSTM + Encoder-Decoder architecture, experimenting with both frozen and fine-tuned embeddings, aiming to discover and understand what works best.

In all proposals, we applied evaluation using different accuracy metrics and compared them. Not all techniques work equally well for every task, and while using the model in inference gives a real view of the results, the numbers are also relevant for an objective comparison.

3.4 Classification BERT fine tuning

After experimentation, we observed that in classification, fine-tuning the Clinical-BERT classification head slightly outperformed classical techniques regarding context understanding, although lightweight Shallow ML models proved to be surprisingly effective and fast.

3.5 Custom summarisation Deep Learning: LSTM

In summarisation, extractive techniques (TextRank) produced grammatically correct but sometimes disjointed results. Conversely, the Encoder-Decoder (LSTM) architecture with adjusted embeddings managed to generate more fluid (abstractive) summaries, though with the added challenge of controlling the "hallucination" of data not present in the original text.

The correction of the embeddings regarding Deliverable 2 proved crucial for improving the convergence of the neural models.

4 Fourth Delivery: Transformers

After receiving the feedback from past deliverables and searching for the best way of approaching each issue, we decided to proceed with the task. The focus this time stayed in the development of models that respecting our hardware limitations, got near to what is being done in the real world, transformers and attention based models for NLP. Therefore, the main objective has been the solution of previous issues and the fine tuning of transformers based models from hugging face, for each task.

4.1 Corrections

Third deliverable corrections included the implementation of Bidirectional-LSTM based custom architecture model we promised for the previous submission and the proper ROUGE evaluation metrics usage for our custom summarisation architecture.

4.2 Transformers

Regarding the inherent tasks of the second iteration / 4th delivery, we applied attention and transformers based models to classification and summarisation tasks, comparing the difference to non attention based approaches and the efficiency of training partial weights or full model weights. As extra work

exploring out personal interests in natural language processing, we fine tuned a decoder only hugging face model for subsequent home server deployment.

Our decision to use transformers was driven by their status as the current state-of-the-art in natural language processing. Unlike recurrent neural networks (RNNs) or LSTMs, which process data sequentially, transformers process the entire input sequence simultaneously using attention mechanisms.

The attention mechanism allows the model to dynamically weigh the importance of different words, not taking into account their distance. In a clinical context, this is crucial, a symptom mentioned at the beginning of a consultation may be directly relevant to a diagnosis made at the end. This way, we can accurately generate summaries and classify them into correct labels.

Rather than training these massive architectures from scratch, which requires big amounts of data and hardware, we adopted a pretrained model fine tuning strategy. This involves taking a pre-trained model (which already understands the structure of language) and training part or all weights on our specific medical dataset.

We experimented with freezing specific layers of the network and updating only a subset of parameters / heads. This approach is more efficient and reduces the risk of catastrophic forgetting, where the model loses its pre-learned general knowledge.

Using these methods we expected to outperform our Shallow ML and LSTM baselines, specifically in generating more coherent summaries and achieving higher accuracy in dialog class classification.

4.3 Custom Classification Deep Learning: Bi-LSTM

Given that in the previous delivery we did not apply Bi-LSTM and fine-tuned a model instead, we will complete the process as we said we would. Here, you can find the process followed to build a custom architecture based on LSTM in order to build a Bi-LSTM for a classification task.

LSTM, applies 3 gates to avoid the vanishing gradients problem that appear in recurrent neural networks (RNN), that way we don't lose the long-term dependencies, the past information, and we are able to understand the whole context.

Famous models as ELMo (Embeddings from Language Models) are based on bidirectional LSTM architectures, but the training time, the datasets and the parameters/weights they train are much larger than the one we have here, but this structure is the base for that famous technology.

Our architecture of selection is a bidirectional LongShortTermMemory model. The reason is the core of LSTM models, which are characterized by

being able to remember the beginning of the text and not miss relevant information. We used this type of models in the past when training a monetary currency price anomaly detector, developed by the same team and publicly available.

Let's get into the architecture:

Custom Bi-LSTM architecture, focused in dialog understanding (for our classification task)

Architecture:

1. Embedding layer
2. Spatial Dropout (for regularization)
3. First Bidirectional LSTM layer
4. Batch Normalization
5. Second Bidirectional LSTM layer
6. Global Max Pooling
7. Dense layers (Dropout)
8. Output layer (Softmax)

The embedding layer acts as a dictionary for the model, giving the model understanding and the relation between words. The spatial dropout acts as a filter, helping the model not to rely on specific words and overfitting with them. The Bidirectional LSTM is the core of our model, it reads the model in both directions. Batch normalization is used to standardize the data that goes through the network. The second Bi-LSTM reads the text again, once the first pass understands the context, this one gives us a deeper understanding of the reading. The dense layer is used to make a decision in respect of the weights and a dropout is added not to overfit, as explained before at spatial dropout.

The final decision is taken at the output layer, which contains a “softmax” function, the list of decisions available in different probabilities. Those probabilities are the confidence our model has on a specific answer.

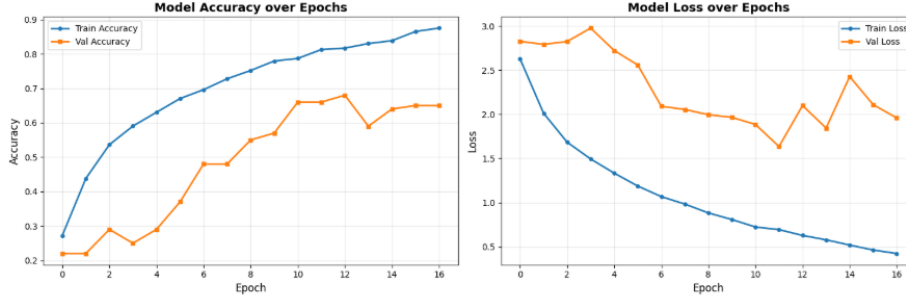


Table 1: Detailed Classification Results and Metrics

Class	Precision	Recall	F1-Score	Support
ALLERGY	0.3333	0.2500	0.2857	4
ASSESSMENT	0.0000	0.0000	0.0000	4
...	...	...	...	...
Accuracy			0.6600	100
Macro Avg	0.2671	0.3014	0.2701	100
Weighted Avg	0.5993	0.6600	0.6119	100

With these metrics, we can see that even if the model achieves progress through training, minimizing the validation loss, and maximizing the accuracy, the model does not achieve to perform correctly. Having great confidence to make an incorrect decision is not the way to go. We consider that the reason for the result may have been the class imbalance, as some classes appear more than others, and training with our training sample may have made us go this way. Therefore, fine tuning a model trained with more data in a more generalized approach using our clinical context may be the way to go. We will see with the fine tuning of distilBERT (uncased).

4.4 Transformers based classification: distilbert-base-uncased

BERT is a transformers based encoder-only model, great at "reading" and labeling (Classification).

As we saw at the bidirectional LSTM, training a model with out data alone was not enough to train a classification model, as the training dataset was greatly imbalanced, we believe that showing our context to a great model based on transformers as distil bert is, will be the way to go.

For this experiment, we used the Google Colab extension for VSCode to access cloud-based hardware acceleration (T4 GPU). This approach was

needed by temporal hardware limitations and demonstrated a viable workflow for resource-constrained development (as in our case).

After preparing the data and training for 50 maximum epochs if the early stopping did not trigger (which did in both cases), we got the following metrics and conclusions from them.

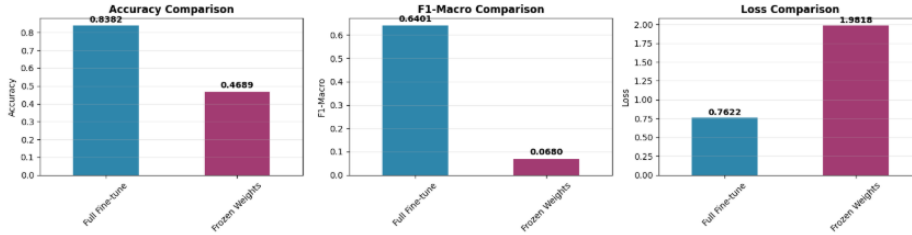


Table 2: Model Performance Comparison

Strategy	Accuracy	F1 Macro	Loss
Full Fine-tune	0.8382	0.6401	0.7622
Frozen Weights	0.4689	0.0680	1.9818

Table 3: Performance Analysis (Full - Frozen)

Metric	Difference	Interpretation
Accuracy	+0.3693	Significant gain in correct predictions.
F1 Macro	+0.5721	Massive improvement in class-balanced performance.

Table 4: Inference Test Results

Dialog	Full Model	Conf.	Frozen Model	Conf.	Interpretation
1	GENHX ✓	0.998	GENHX ✓	0.370	Correct Match
2	ROS ✓	0.920	GENHX ×	0.277	Frozen Incorrect
3	PASTMEDICALHX ✓	0.995	FAM/SOCHX ×	0.331	Frozen Incorrect

Given the shown results, even if we suspected that the frozen weights would only give up a little accuracy for the lower training cost, we can conclude that the Full Fine-Tuning strategy is overwhelmingly superior to the frozen weights approach for the distilbert model classification task.

The results indicate that the pre-trained representations alone were insufficient to properly generalize, requiring the adjustment of all model weights to achieve acceptable performance.

The gap in F1 Macro (+0.5721) suggests that the Frozen Weights model likely collapsed into predicting the majority class or failed to distinguish between specific labels, where the fully fine tuned model successfully learned.

The inference test highlights a critical reliability issue with the frozen model. Even when it predicted correctly the first dialog, its confidence was always low, compared to the near-certainty of the full model, which correctly predicted all classes with great confidence.

With an accuracy below 50 percent and a loss nearly three times higher than the alternative, the Frozen Weights strategy would not be viable for deployment. The full fine tune, despite the slightly higher (3 epoch) computational cost during training, is the only logical path.

Comparing it to the previous delivery, where we fine tuned the classification head of BioClinical bert, distil bert outperforms the previous implementation, inference test results, confidence and metrics, fully or with frozen weight fine tuning. This fact states the superiority of distilBERT if we give it time to improve, when even both being based on transformers and bio-ClinicalBERT almost doubling the amount of parameters, was only given 3 epochs to learn. This shows us that even if the amount of parameters is higher, the training time has a lot to do with its quality. In addition, both models outperformed our custom architecture based on bidirectional LSTM model, confirming our assumption that correlated the class imbalance of the dataset to the overfitting on some classes (failing to classify everything that was not classified as what it had seen most).

Distilbert: 66 Million Parameters - 0.998 (Correct Prediction)

BioClinicalBERT: 110 Million Parameters - 0.924 confidence (Correct Prediction)

Apart from the model comparison, as mentioned at the beginning, we also used this notebook to try the google colab extension that provides virtual hardware in case your computer does not bring one, in our case, we tested it in a laptop that did not bring a GPU, and used the free tier T4. Even if it was not essential this time, free virtual hardware may come handy if working far from home or out of resources in the future. The functionality is controlled by the boolean at the first code cell and controls the dataset import, persistence and data visualization (which requires an API key for the weights and biases service).

4.5 Transformers based summarisation: T5-small

The objective of this task was to develop and evaluate an advanced solution for text summarization by applying the transformer-based architectures as we did in classification. Continuing from our previous experiments with shallow machine learning and custom deep learning architectures. This final phase focuses on fine-tuning a pre-trained llm, which we believed would achieve the best performance in comparison to other approaches.

For this experiment, we used the t5-small hugging face model, a lightweight variant of the T5 (Text-to-Text Transfer Transformer) model. T5 is well-suited for our problem, therefore we had high expectations.

The goal of this analysis was not only to achieve high metric scores but to investigate the trade-off between model performance and computational efficiency. We aimed to compare if fully training a model’s weights instead of fine tuning it partially was viable.

After training for a maximum of 50 epochs as in the previous approach, if not stopped by the early stopping, we received the following metrics and drew conclusions.

Frozen Backbone Lora Fine Tune: 10 Epochs before early stop Full Model Fine Tune: 10 + 5 Epochs before early stop (we used the frozen backbone and continued from there in order to not retrain already trained weights)

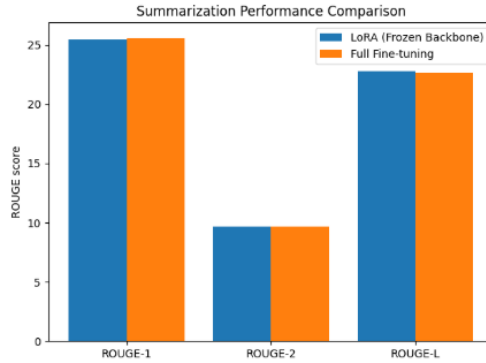


Table 5: ROUGE Score Comparison

Strategy	Rouge 1	Rouge 2	Rouge L
Frozen Backbone	25.4643	9.6973	22.7828
Full Fine-tune	25.6086	9.6415	22.6527

Even if the resulting summaries are not so close to the one provided at the validation set, they are actually pretty well and correct. They seem to go in the right direction and we think that unlocking the resulting token amount would only result in a complete summary of the provided context. Therefore we are so happy with the inference results!

Both models answer the same, they summarise the validation samples with the same answers, therefore, we can assume that in the real world, they perform equally well. This could be changed adding a probabilistic nature to the model, developing a non deterministic model. But it is not a problem at all, it is just a feature of our approach.

We can't really compare it to the custom deep learning architecture that we developed in the previous delivery, as ROUGE metric was not used, but we can do it with the shallow ML approach, where the metrics show a much better performance compared to it (as we can expect from a model trained with more data and fine tuned to our context and needs).

The metrics show us that the additional training of the full model gave us almost intangible improvements over the frozen backbone approach. Given the similarities in ROUGE scores and identical inference outputs, the frozen backbone strategy is more efficient as it achieves comparable performance with significantly fewer trainable parameters.

4.6 Decoder Only Generative LLM fine tuning

Even if it was not on our initial goal we aimed to go even further. We tried encoder only models for text classification, and encoder-decoder models for text summarisation, all transformer based and usage of attention, therefore, in order to finish the full experimentation, we will train a decoder only model that acts as the doctor and helps the incoming patients. In case anyone tries to get real medical opinion from it, this model is not professional, the clinical accuracy and strictness of the data it was finetuned with has not been checked as we are not trained in the clinical sector and we completely discourage and will not take responsibility for the usage of this model as real clinical support, as it is an educational purpose developed model and its decisions should not be taken seriously. Even if we will do our best training it.

The purpose of the model is to be deployed in a raspberry 5 16 gb home server as an API service for other programs (twitter bot and web based chat bot), therefore we aim to build a complete deployment pipeline from the model we create here. The pipeline would look like this: the model running in a containerized server from ollama, and accessible via Fast-API.

As the hardware available for inference is able to run small models, as gemma2 with 2 b parameters, we believed that it would be the way to go. The problem arrived when we started training the model, and realized that only training between 0.5 and 3 percent of the parameters using PEFT lora, 1 percent of 2.000 million parameters is still too much for our hardware, and it would take complete days of total VRAM usage training, and we decided to pivot to a realistic approach that consisted in the fine tuning of a 87 million parameter model, distilGPT2. During the training of the model, which used the whole 50 epochs without early stopping, even if the improvement after 25 was not as much, we saw that the model learnt to generate the whole conversation, taking the role of the doctor, guests, second doctor and patient. This issue indicated that the preprocessing led the model to overfit, and

literally talk with itself taking different roles. Moreover, we decided to apply a penalty for when the model got stuck in a single theme, because sometimes it started asking the patient if each part of the body was injured, one by one. Finally, the model learnt to answer the questions asked given a context from previous conversations (if stated) and proved to be useful, taking into account the generalization a model of this size can achieve.

As we mentioned, this generative model will be used to be deployed in ollama and available through an API in a personal portfolio behind an authorization wall. Stated in the repository when available.

5 Contributions of the Project to our Knowledge and Experience

Through the four deliveries we have developed for this project, we have followed all the steps of a clinical NLP system, starting from basic preprocessing and ending with the deployment of generative models. On first sight, we discovered that for classification tasks with limited datasets, ShallowML approaches like TF-IDF combined with SVMs work better or at least as good as more complex neural networks. However, to get the deep semantic context required in this exact domain, which is a complex and technical domain, we confirmed that models based on transformers such as DistilBERT are better suited. Our experiments showed that fully fine-tuning these models are significantly better than using frozen embeddings, as it allows the model to adapt the knowledge specifically to patient-doctor dialogues. We also faced a significant problem with the dataset, because the classes were imbalanced. We saw that architectures trained from scratch were affected by this imbalance, biasing the predictions to the classes that had more examples. In contrast, pre-trained transformer models demonstrated higher robustness. Their prior knowledge allowed them to generalize better with fewer examples per class.

We also learned to adapt the tasks to work with the hardware limitations we have, working with low memory graphic-cards configuring the batch size and gradients, CPU only environments, selecting proper models by parameter size and checking if our models overfit and how to solve common issues. This was a problem during the training of the Bi-LSTM architecture, when we had some CUDA problems, we successfully changed the training process to the CPU to ensure the working of these modules.

This project covered a complete NLP project, from basic feature engineering to the deployment of a generative AI system. Going further from

just having it in Jupyter notebooks, and trying to deploy a decoder-only model on a Raspberry Pi. This work has expanded our toolset to include the Hugging Face ecosystem, TensorFlow, and advanced error analysis metrics. We have changed our scope from just simply running code to making engineering decisions, choosing the right architecture based on data constraints and deploying solutions in real world environments.

6 References

An Empirical Study of Clinical Note Generation from Doctor-Patient Encounters - by Ben Abacha, Asma and Yim, Wen-wai and Fan, Yadan and Lin, Thomas - 2023

Publicly available clinical BERT embeddings - Alsentzer, Emily and Murphy, John R and Boag, Willie and Weng, Wei-Hung and Jin, Di and Naumann, Tristan and McDermott, Matthew - 2019

DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter - by Sanh, Victor and Debut, Lysandre and Chaumond, Julien and Wolf, Thomas - 2019

Exploring the limits of transfer learning with a unified text-to-text transformer - by Raffel, Colin and Shazeer, Noam and Roberts, Adam and Lee, Katherine and Narang, Sharan and Matena, Michael and Zhou, Yanqi and Li, Wei and Liu, Peter J - 2020

ROUGE: A package for automatic evaluation of summaries - by Lin, Chin-Yew - 2004

Deep contextualized word representations - by Peters, Matthew E and Neumann, Mark and Iyyer, Mohit and Gardner, Matt and Clark, Christopher and Lee, Kenton and Zettlemoyer, Luke - 2018

Long short-term memory - by Hochreiter, Sepp and Schmidhuber, Jürgen - 1997